

Ejercicio 03 - Metodologías ágiles

Raúl Ángel Lago Castro

Introducción

En este trabajo voy a explicar cinco metodologías ágiles que hemos visto en clase: SCRUM, Kanban, SCRUMban, Lean y Extreme Programming. También voy a comparar estas metodologías con las tradicionales (como el modelo en cascada).

Antes de entrar en cada una por separado, creo que hay que explicar un poco la idea general de lo que es "ágil". Las metodologías ágiles nacen porque los modelos antiguos de desarrollo de software no se adaptaban bien a los cambios. Con el modelo en cascada, por ejemplo, tenías que tener clarísimo desde el principio todo lo que ibas a hacer, y si luego cambiaba algo era un problema grande. Lo ágil va de otra manera: se trabaja en partes pequeñas y cortas, y al terminar cada una de esas partes ya tienes algo que funciona, aunque sea una versión sencilla. Así puedes ir corrigiendo el rumbo según haga falta, en vez de darte cuenta de los fallos al final de todo el proyecto. Algo importante en programación porque generalmente por lo que voy viendo te adentras en lo desconocido en la mayoría de ocasiones.

SCRUM

SCRUM es probablemente la metodología ágil más conocida y la que más se usa hoy en día. Se basa en dividir el proyecto en periodos cortos de tiempo que se llaman sprints, que suelen durar entre una y cuatro semanas. Al terminar cada sprint tiene que haber algo ya construido y que funcione, no solo una parte a medias.

En SCRUM hay tres roles principales. Está el Product Owner, que es quien decide qué hay que hacer y en qué orden según lo que necesite el cliente o el proyecto. Está el Scrum Master, que no es exactamente un jefe sino más bien la persona que ayuda al equipo a que el proceso funcione bien y quita los obstáculos que puedan surgir. Y luego está el equipo de desarrollo, que es quien construye el producto en sí.

También se usan un par de listas para organizar el trabajo: el Product Backlog, que es la lista con todo lo que hay que hacer en el proyecto entero, y el Sprint Backlog, que es la parte de esa lista que el equipo coge para hacer en el sprint que toca.

Por último, SCRUM tiene varias reuniones fijas. Al empezar el sprint se hace una reunión de planificación para decidir qué se va a hacer. Cada día se hace una reunión muy corta (el daily) donde cada uno cuenta en qué está trabajando y si tiene algún problema. Al terminar el sprint

se hace una revisión para enseñar lo que se ha hecho, y después una retrospectiva donde el equipo habla de cómo ha ido el trabajo y qué se puede mejorar para el siguiente sprint.

Kanban

Kanban es otra metodología ágil, pero a diferencia de SCRUM no trabaja con sprints ni con tiempos cerrados. Se basa en un tablero visual dividido en columnas, normalmente algo como "Por hacer", "En proceso" y "Hecho", aunque se pueden añadir más columnas según lo que necesite el equipo. Cada tarea es una tarjeta que se va moviendo de columna según va avanzando.

Una de las ideas más importantes de Kanban es el límite de trabajo en curso, es decir, que no se pueden tener demasiadas tareas abiertas a la vez en la misma columna. Esto obliga a terminar las cosas antes de empezar otras nuevas, y ayuda a ver rápido dónde se está atascando el trabajo.

A diferencia de SCRUM, en Kanban el flujo de trabajo es continuo. No hay que esperar a que acabe un sprint para meter una tarea nueva, se puede añadir en cualquier momento según va haciendo falta. Por eso se suele decir que Kanban es más flexible para proyectos donde las prioridades cambian mucho o donde entran peticiones nuevas constantemente, como puede ser un equipo de soporte técnico.

SCRUMban

SCRUMban es, como su nombre indica, una mezcla entre SCRUM y Kanban. Coge cosas de los dos: por un lado mantiene algunas reuniones de SCRUM, como las reuniones diarias o las retrospectivas, pero por otro lado usa el tablero visual de Kanban y el flujo continuo de tareas, sin estar tan atado a sprints cerrados.

Esta metodología suele usarse en equipos que ya trabajaban con SCRUM pero que necesitan más flexibilidad, o que están cambiando poco a poco de SCRUM hacia Kanban. También es útil en proyectos de mantenimiento, donde no siempre tiene sentido planificar sprints enteros porque las tareas que van llegando son más pequeñas e imprevisibles.

Lean

Lean es un poco diferente de las otras porque en realidad no nació en el mundo del software. Viene de la industria, en concreto del sistema de producción de Toyota, y después se adaptó al desarrollo de software. La idea principal de Lean es eliminar todo lo que no aporta valor al producto final, es decir, quitar de en medio todo lo que sea desperdicio de tiempo o de recursos.

Algunos de los principios de Lean son: eliminar lo que no sirve, entregar lo antes posible al cliente, dejar que decida el propio equipo en vez de que todo venga impuesto desde arriba, y mejorar continuamente el proceso de trabajo. En el fondo comparte bastante filosofía con las demás metodologías ágiles, pero pone más el foco en no malgastar esfuerzo en cosas que luego no se van a usar o que no le importan al cliente.

Extreme Programming (XP)

Extreme Programming, o XP, es la metodología más centrada en la parte técnica y en cómo se escribe el código, no tanto en cómo se organiza el equipo o las reuniones. Busca mejorar la calidad del software y la capacidad de adaptarse a los cambios llevando al extremo algunas prácticas que ya se usaban antes, de ahí el nombre.

Algunas de sus prácticas más conocidas son la programación en parejas, donde dos personas programan juntas en el mismo ordenador (una escribe y la otra revisa en el momento), y el desarrollo guiado por pruebas, que consiste en escribir primero la prueba que tiene que pasar el código y después escribir el código necesario para que esa prueba se cumpla. También da mucha importancia a la integración continua, es decir, ir juntando el código de todo el equipo muy a menudo para detectar fallos cuanto antes, y a tener siempre comunicación directa con el cliente.

Comparación con las metodologías tradicionales

Las metodologías tradicionales, como el modelo en cascada, funcionan por fases que van una detrás de otra: primero se analiza todo el proyecto, después se diseña, después se programa, después se prueba y por último se mantiene. El problema de este modelo es que no se puede volver atrás fácilmente. Si en la fase de pruebas te das cuenta de que algo del diseño estaba mal planteado, el cambio sale muy caro porque ya se ha avanzado mucho sobre esa base.

Otro problema del modelo en cascada es que hace falta tener clarísimos todos los requisitos desde el principio, y en la práctica eso casi nunca pasa. El cliente muchas veces no sabe exactamente lo que quiere hasta que no ve algo funcionando, y con cascada no ve nada hasta que el proyecto está casi terminado.

Las metodologías ágiles solucionan bastante estos problemas. Al trabajar en partes pequeñas y entregar versiones funcionales todo el rato, el cliente puede ir viendo el progreso real y pedir cambios antes de que sea demasiado tarde. Esto reduce el riesgo de terminar el proyecto entero y que no sea lo que el cliente necesitaba. También ayuda mucho la comunicación constante dentro del equipo, cosa que en cascada no se cuida tanto porque cada fase la suele llevar gente distinta y se pasa el trabajo como una cadena de producción.

Como ventaja de lo ágil yo destacaría sobre todo tres cosas: se detectan los errores antes porque se prueba constantemente, se puede cambiar de dirección sin que cueste un drama porque el proyecto no está cerrado desde el minuto uno, y el equipo trabaja más motivado porque ve resultados reales cada poco tiempo en vez de estar meses sin ver nada construido.

Qué metodología elegiría yo

Si tengo que elegir una metodología para trabajar en un grupo, me quedo con Kanban. SCRUM también me parece bien, pero Kanban me convence más por lo sencillo y visual que es.

Lo que más me gusta es el tablero con las columnas de tareas por hacer, en proceso y terminadas. Con solo mirarlo un segundo ya sabes cómo va el proyecto entero, sin tener que preguntar a nadie ni esperar a ninguna reunión. A mi me gusta las cosas directas para trabajar, para hablar el chigre.

También me parece muy buena idea lo de limitar el número de tareas que puede haber a la vez en cada columna. Si una columna empieza a llenarse de tarjetas, es una señal clara de que algo se está atascando ahí, y se puede reorganizar enseguida sin tener que esperar a que acabe un sprint para darse cuenta del problema. Eso hace que los cuellos de botella se vean y se arreglen mucho más rápido.

Otra cosa que me convence es que el trabajo en Kanban es continuo, no está dividido en sprints cerrados como en SCRUM. Si surge una tarea nueva o cambia una prioridad, se puede meter en el tablero en el momento, sin tener que esperar a que termine la iteración actual. Me parece más realista para muchos proyectos, porque las prioridades casi nunca se quedan quietas durante semanas.

Referencias externas

<https://www.youtube.com/watch?v=AAZtxEFWqDI>

<https://www.youtube.com/watch?v=Smf2BKwiBmM>